

Sanket Ajay Chopade

DevOps Engineer

Interview Preparation Guide

AWS · Kubernetes · Terraform · Jenkins · Docker · Prometheus

GII Global Infotech India Pvt. Ltd.

10 Categories · 30 Questions · Structured Answers

chopade.sanket@outlook.com

[linkedin.com/in/sanketchopade07](https://www.linkedin.com/in/sanketchopade07)

github.com/Sanket006

This guide contains 30 interview questions across 10 categories, each with structured answers tied directly to your resume and the GII Global Infotech job description. Every answer includes a concrete reference to your internship at Hisan Labs or your projects (ApnaKart, Flight Reservation). Use the interview tips at the end of each answer to calibrate how much depth to go into. Practise each answer out loud — not by reading, by speaking.

1. Kubernetes & Container Orchestration

Q1. Explain your Kubernetes architecture end-to-end.

ARCHITECTURE

- Control plane (API server, etcd, scheduler, controller manager) managed by AWS in EKS; worker nodes owned by you.
- In ApnaKart: 10 Spring Boot microservices each deployed as separate Deployments across EKS worker nodes.
- ALB Ingress Controller acts as single entry point — routes traffic to ClusterIP services internally.
- EKS nodes placed in private subnets; ALB in public subnet — network isolation by design.

KEY PRIMITIVES USED

- Deployments for stateless services with rolling update strategy.
- HPA (Horizontal Pod Autoscaler) — auto-scaled on CPU/memory thresholds, absorbed 5× peak load in Flight Reservation.
- RBAC — least-privilege service accounts per microservice.
- ConfigMaps for non-sensitive config; Secrets for credentials — never hardcoded in images.
- Liveness and Readiness probes on every Deployment for self-healing.

OUTCOME

- 35% fewer deployment failures, 99.9% system availability.

Interview tip: Draw the architecture in your head — ALB → Ingress → Service → Pod. Mention database-per-service isolation as a design decision.

Q2. Pod is in CrashLoopBackOff. Walk me through your debugging process.

STEP-BY-STEP

- Step 1: `kubectl get pods` — confirm status and restart count.
- Step 2: `kubectl describe pod` — check Events section (OOMKilled, image pull error, probe failure).
- Step 3: `kubectl logs --previous` — read logs from the crashed instance, not the current restart.
- Step 4: Check resource limits — OOMKilled means container exceeded memory ceiling; increase limit or fix memory leak.
- Step 5: Check liveness probe config — misconfigured path/timeout kills healthy pods in a loop.
- Step 6: Check image tag validity and ECR authentication.
- Step 7: Verify all required env vars, ConfigMaps, and Secrets are mounted correctly.

FROM YOUR EXPERIENCE

- In ApnaKart, enforced resource limits on all Deployments — OOMKilled was the most common CrashLoop cause.
- Self-healing restarts via Deployments mean K8s keeps trying — fix root cause, not the symptom.

Interview tip: `kubectl describe` first, always. Events section resolves 80% of cases before you even look at logs.

Q3. Difference between StatefulSet and Deployment?

DEPLOYMENT

- For stateless apps — pods are interchangeable, no stable identity.
- Pods get random names on restart; no persistent storage by default.
- Used for all 10 Spring Boot microservices in ApnaKart.

STATEFULSET

- For stateful apps — each pod has a stable network identity (pod-0, pod-1) and persistent volume.
- Pods start and terminate in order; useful for databases, Kafka, Zookeeper.
- RDS was used externally in ApnaKart instead of in-cluster StatefulSets — AWS-managed DB is more reliable.

Interview tip: Follow up ready: 'Why didn't you use StatefulSet for your DB?' — Answer: RDS gives managed backups, multi-AZ, automated failover. Running DB in K8s adds operational complexity without benefit.

Q4. How do you perform zero-downtime deployments?

ROLLING UPDATE (DEFAULT)

- Kubernetes replaces pods incrementally — new pods come up, old come down.
- maxSurge and maxUnavailable control speed vs. stability trade-off.
- Readiness probes ensure new pod serves traffic only when ready.

BLUE-GREEN

- Two identical environments (blue = live, green = new version).
- Deploy to green, run smoke tests, then switch ALB target group to green.
- Implemented at Hisan Labs — 98% zero-downtime release success rate.
- Instant rollback: switch ALB back to blue if issues detected.

Interview tip: Mention both strategies and when you'd use each. Blue-green = safer, higher cost. Rolling = lower cost, longer rollout window.

2. CI/CD (Jenkins + GitHub Actions)

Q1. Explain your CI/CD pipeline in detail.

PIPELINE STAGES

- Trigger: GitHub push/PR webhook fires Jenkins job automatically.
- Stage 1 — Build: Maven compiles Spring Boot app and runs unit tests.
- Stage 2 — Code Quality: SonarQube static analysis; pipeline fails if critical issues found.
- Stage 3 — Image Build: Docker multi-stage build reduces image size; Trivy scans for CVEs.
- Stage 4 — Push: Docker image tagged (commit SHA + latest) and pushed to AWS ECR.
- Stage 5 — Deploy: kubectl apply triggers rolling update on EKS; deployment status monitored.

KEY DESIGN DECISIONS

- Declarative Jenkinsfile stored in repo — pipeline-as-code, version controlled.
- Credentials in Jenkins Credentials Manager — never in Jenkinsfile or environment vars.
- Webhook ensures fully automated pipeline with zero manual steps.

RESULT

- Deployment time: 45 minutes → 10 minutes (77% reduction).
- Release frequency: increased 3×.
- Zero critical vulnerabilities in production due to Trivy + SonarQube gates.

Interview tip: Interviewers will probe: 'What caused the 45-minute bottleneck?' Be ready to say — serial execution, large unoptimised images, no layer caching. Parallelised stages and multi-stage builds fixed it.

Q2. Jenkins build fails in production. How do you debug?

DIAGNOSIS

- Open Jenkins Console Output — read the exact error, never guess.
- Identify which stage failed: Build → Test → Quality Gate → Docker → Deploy.

PER-STAGE FAILURES

- Build stage: dependency download failed (network/Nexus), compilation error (code issue).
- Test stage: unit test failure — check if flaky test or real regression.
- SonarQube gate: code quality threshold breached — review the flagged issues.
- Docker stage: base image pull rate limit (DockerHub), disk full on agent.
- Deploy stage: kubeconfig wrong context, EKS API unreachable, image tag mismatch.

ROLLBACK

- `kubectl rollout undo deployment/` — instantly reverts to previous stable ReplicaSet.
- Blue-green: switch ALB target group back to previous environment.

Interview tip: Show a systematic approach — stage-by-stage, not trial and error. Mention that rollback is immediate in your setup.

Q3. How do you implement rollback in your pipeline?

KUBERNETES ROLLBACK

- Kubernetes retains previous ReplicaSet — `kubectl rollout undo deployment/` restores it instantly.
- `kubectl rollout history` shows all revisions for selective rollback.

BLUE-GREEN ROLLBACK

- Blue (old) environment stays live until green is validated.
- Switch ALB target group back to blue — near-instant, no redeployment needed.
- Used this at Hisan Labs — 98% zero-downtime release success rate.

PIPELINE-LEVEL GATE

- Post-deploy health check (`curl` readiness endpoint) built into Jenkinsfile.
- If health check fails, pipeline triggers automatic rollback before marking deploy success.

Interview tip: Key distinction: `kubectl rollout undo` is fast but limited to previous version. Blue-green gives you full control and immediate reversal without any Kubernetes operation.

3. AWS & Cloud Infrastructure

Q1. Difference between Security Group and NACL?

SECURITY GROUP

- Instance-level firewall — attached to EC2, RDS, ALB, Lambda.
- Stateful: return traffic automatically allowed without explicit rule.
- Allow rules only — no explicit deny.

NACL (NETWORK ACL)

- Subnet-level firewall — all resources in a subnet inherit it.
- Stateless: must explicitly allow both inbound and outbound.
- Both allow and deny rules, evaluated in order by rule number.
- First line of defence before traffic reaches Security Group.

YOUR USAGE

- Terraform provisioned VPCs with public/private subnets — ALB in public, EKS nodes and RDS in private.
- NACLs block unwanted IP ranges at subnet level; SGs restrict port access at resource level.

Interview tip: Classic follow-up: 'Port 443 allowed in SG but NACL denies it — what happens?' Answer: blocked. NACL evaluated first at subnet boundary.

Q2. EC2 instance reachable internally but not publicly. Why?

CHECKLIST

- Security Group: Is inbound port 80/443 open for 0.0.0.0/0?
- NACL: Is inbound and outbound traffic allowed for required ports?
- Subnet: Is the EC2 in a public subnet with auto-assign public IP enabled?
- Internet Gateway: Is IGW attached to the VPC and route table has 0.0.0.0/0 → IGW?
- Route table: Is the subnet's route table associated with the IGW route?
- Elastic IP: If using EIP, is it correctly associated with the instance?

MOST COMMON CAUSE

- Instance placed in private subnet by mistake — no direct internet route.
- Security group missing inbound rule for 0.0.0.0/0 on the application port.

Interview tip: Work outward-in: Internet → IGW → Route Table → NACL → SG → Instance. This systematic approach impresses interviewers.

Q3. How would you reduce AWS costs?

COMPUTE

- Use Spot Instances for non-critical workloads (CI runners, dev environments) — up to 90% saving.
- Reserved Instances for predictable baseline workloads — up to 72% vs on-demand.
- Right-size EC2/EKS nodes — CloudWatch metrics reveal over-provisioned instances.

STORAGE

- S3 lifecycle policies — transition objects to Glacier after 30 days, delete after 90.
- Enable S3 Intelligent-Tiering for unpredictable access patterns.

ARCHITECTURE

- HPA prevents over-provisioning by scaling pods dynamically — not running max capacity 24/7.
- CloudFront caches static assets — reduces origin (EC2/ALB) request volume and data transfer costs.
- Lambda for event-driven tasks instead of always-on EC2.

Interview tip: Mention that your Terraform modules enable environment parity — dev spins down overnight via scheduled scaling, saving ~60% on non-prod costs.

4. Terraform & Infrastructure as Code

Q1. Explain your Terraform workflow.

CORE COMMANDS

- terraform init — downloads provider plugins (AWS), initialises backend (S3 remote state).
- terraform plan — dry run showing creates/updates/destroys. Never skip this in production.
- terraform apply — executes the plan. Use -auto-approve only in CI with safeguards.
- terraform destroy — tears down all managed resources (used for ephemeral environments).

STATE MANAGEMENT

- Remote state in S3 with DynamoDB state locking — prevents concurrent applies.
- If two engineers apply simultaneously, second gets a lock error and waits.
- State corruption recovery: terraform state list → terraform state rm → terraform import.

YOUR USAGE (APNAKART)

- Modular structure: separate modules for VPC, EKS, RDS, IAM, S3, ALB, CloudFront.
- Modules are reusable across dev/staging/prod with different variable files.
- Reduced provisioning time by 40%; fully repeatable deployments.

Interview tip: Interviewers often ask about drift detection. Answer: terraform plan shows drift between state and real infra. Automate it in CI to catch console changes.

Q2. Two engineers modified infrastructure simultaneously. What happens?

WITH DYNAMODB LOCKING (CORRECT SETUP)

- First engineer's apply acquires a lock in DynamoDB table.
- Second engineer's apply sees the lock and either waits or errors with: 'Error acquiring state lock'.
- Once first apply completes, lock is released and second can proceed.

WITHOUT LOCKING (DANGEROUS)

- Both reads stale state, both apply conflicting changes — last write wins.
- State file becomes inconsistent — resources exist in AWS but not tracked in state.
- Recovery is manual and risky — terraform import for each orphaned resource.

PREVENTION

- Always configure S3 backend with DynamoDB locking — non-negotiable in team environments.
- Use Terraform Cloud or Atlantis for PR-based apply workflows with approval gates.

Interview tip: Show you've thought about team workflows, not just solo usage. Mention Atlantis or Terraform Cloud as the gold standard for team IaC.

5. Docker & Containerisation

Q1. How do you optimise Docker image size?

MULTI-STAGE BUILDS

- Stage 1 (builder): full JDK + Maven to compile and package the app.
- Stage 2 (runtime): only JRE + the compiled JAR — strips all build tooling.
- Used in Flight Reservation: 60% image size reduction across 5 services.

OTHER TECHNIQUES

- Use minimal base images: eclipse-temurin:17-jre-alpine instead of full Ubuntu.
- Combine RUN commands to reduce layers — each RUN creates a new layer.
- Add .dockerignore to exclude target/, node_modules/, .git from build context.
- Avoid copying unnecessary files — only COPY what the runtime needs.

WHY IT MATTERS

- Smaller images = faster pulls in CI, faster Kubernetes pod starts, smaller attack surface.

Interview tip: Lead with the 60% reduction stat from your resume. Concrete numbers always beat abstract explanations.

Q2. CMD vs ENTRYPOINT in Dockerfile?

ENTRYPOINT

- Defines the executable that always runs — cannot be overridden without `--entrypoint` flag.
- Use for the main process: `ENTRYPOINT ["java", "-jar", "app.jar"]`

CMD

- Provides default arguments to ENTRYPOINT, or default command if ENTRYPOINT is absent.
- Can be overridden easily: `docker run myimage --spring.profiles.active=prod`

COMBINED USAGE

- `ENTRYPOINT ["java", "-jar", "app.jar"]` + `CMD ["--server.port=8080"]`
- CMD provides overridable defaults; ENTRYPOINT locks the executable.

Interview tip: For Spring Boot microservices, always use ENTRYPOINT for the java command and CMD for overridable JVM flags or Spring profiles.

6. Monitoring & Observability

Q1. How do you monitor your applications in production?

YOUR OBSERVABILITY STACK

- Prometheus: scrapes metrics from `/actuator/prometheus` endpoints on Spring Boot apps and node exporters.
- Grafana: dashboards for RED metrics — Request rate, Error rate, Duration (latency) per service.
- Datadog: APM for distributed tracing across microservices; log aggregation and alerting.
- AWS CloudWatch: EKS node metrics, RDS performance, ALB request counts, Lambda errors.

WHAT YOU MONITOR

- Infrastructure: CPU, memory, disk I/O on EKS nodes.
- Application: P95/P99 latency, error rate, throughput per endpoint.
- Kubernetes: pod restart count, pending pods, HPA scaling events, node status.
- Deployments: compare error rate before and after deploy as automated post-deploy check.

RESULT

- MTTR reduced by 30% — proactive alerts caught issues before users reported them.

Interview tip: Key distinction: Prometheus is pull-based (scrapes targets on interval); Datadog is push-based (agent ships data). Interviewers frequently test this.

Q2. CPU spike detected in production. What are your next steps?

IMMEDIATE

- Check Grafana dashboard — confirm spike scope (single pod, all pods, all nodes).
- Check Prometheus alerts — identify which service is affected.
- If spike is on one pod: check if it's a runaway thread (kubectl exec + jstack for JVM apps).

INVESTIGATION

- kubectl top pods — rank pods by CPU consumption.
- kubectl top nodes — check if node is saturated; consider cordoning it.
- Check recent deployments in Jenkins — did a new release cause it?
- Check HPA — is it already scaling out? If not, why? (Check HPA min/max and cooldown period.)

RESOLUTION OPTIONS

- If new deployment caused it: kubectl rollout undo deployment/.
- If traffic spike: HPA should auto-scale — verify thresholds are correctly set.
- If resource leak: fix in code, redeploy with rolling update.

Interview tip: Show a metrics-first mindset. The answer starts with observability data, not guessing. Mention that HPA is your first line of defence against traffic-driven CPU spikes.

7. DevSecOps & Security

Q1. How do you implement security in your CI/CD pipeline?

CODE LEVEL

- SonarQube static analysis in every build — fails pipeline on critical security issues.
- Dependency vulnerability scanning via SonarQube plugins.

CONTAINER LEVEL

- Trivy scans every Docker image before it reaches ECR — zero critical CVEs in production.
- Multi-stage builds minimise image attack surface by excluding build tools from runtime.

INFRASTRUCTURE LEVEL

- IAM roles with least-privilege permissions — each service has scoped role, no wildcard policies.
- Kubernetes RBAC: service accounts with minimal permissions; no cluster-admin in app namespaces.
- Secrets managed via Kubernetes Secrets (base64) or ideally AWS Secrets Manager — never in env vars or code.

Interview tip: DevSecOps = shift security left. Security checks in CI fail fast — cheaper to fix at build time than in production.

Q2. Vulnerability found in a production Docker image. What do you do?

IMMEDIATE

- Assess severity — Critical CVE needs immediate action; Medium/Low can be scheduled.
- Check if the vulnerability is exploitable in your context (network exposure, privileges required).

REMEDIATION

- Update base image to patched version in Dockerfile.
- Rebuild image, run Trivy to confirm vulnerability resolved.
- Push patched image to ECR with new tag.
- Deploy via normal rolling update pipeline — no shortcuts.

PREVENTION

- Enable Trivy in CI pipeline to block vulnerable images before they reach production.
- Schedule weekly base image rebuilds to pick up upstream OS patches.
- Use Amazon ECR image scanning (native) as secondary gate.

Interview tip: Show you treat it as a process, not a one-off panic. Pipeline gates prevent recurrence — that's the key takeaway.

8. Linux & Scripting

Q1. Production server disk is suddenly full. How do you investigate?

STEP-BY-STEP

- `df -h` — identify which filesystem is full (`/`, `/var`, `/tmp`).
- `du -sh /* | sort -rh | head -20` — find largest directories.
- Drill down: `du -sh /var/log/*` to find large log files.
- Check container logs: `docker system df` — Docker layer/volume accumulation is common.
- Check for deleted but open file handles: `lsof | grep deleted` — files deleted but still held open by processes.

RESOLUTION

- Clear old logs: `journalctl --vacuum-time=7d` or truncate specific log files.
- `docker system prune` to remove unused images, stopped containers, unused volumes.
- Implement log rotation (`logrotate`) to prevent recurrence.

Interview tip: Show you know the difference between disk space (`df`) and disk usage (`du`). Mention log rotation as prevention — shows operational maturity.

Q2. What did you automate with Bash scripting at Hisan Labs?

EXAMPLES FROM YOUR EXPERIENCE

- Automated repetitive Linux operations — eliminated 70% of manual tasks.
- Health check scripts: curl endpoints, check HTTP status codes, alert on failure.
- Log rotation and cleanup scripts on EKS nodes to prevent disk fill.
- Environment setup scripts for dev onboarding — consistent toolchain in one command.
- Post-deployment smoke tests: verify all pods are Running, check service endpoints.

WHY BASH OVER PYTHON FOR OPS TASKS

- Bash is universally available on Linux — no runtime to install.
- Python used for complex logic (data parsing, API calls); Bash for file ops and command chaining.

Interview tip: Quantify the automation impact: '70% reduction in manual tasks' from your resume. That's your headline metric — use it.

9. Behavioral & Project Discussion

Q1. Tell me about yourself.

STRUCTURE (90 SECONDS)

- Who you are: 'I'm a DevOps Engineer with hands-on experience in cloud infrastructure, Kubernetes, and CI/CD automation.'
- What you've done: 'At Hisan Labs, I built and owned the entire DevOps stack for a 10-service microservices platform — Terraform-provisioned EKS on AWS, Jenkins pipelines, Prometheus/Grafana observability, and DevSecOps practices.'
- Key result: 'I reduced deployment time by 77%, achieved 99.9% system availability, and eliminated critical vulnerabilities from production.'
- Why DevOps: 'I'm drawn to DevOps because it sits at the intersection of infrastructure, automation, and product delivery — where small improvements in tooling have large downstream impact.'
- Why this role: 'This role aligns with what I've been building toward — production-scale Kubernetes, CI/CD ownership, and cloud-native infrastructure.'

Interview tip: Practise this until it takes under 90 seconds. Do not list technologies — tell a story. The numbers (77%, 99.9%, 70%) are your proof points.

Q2. Describe a production issue you handled. Walk me through it.

SITUATION

- Post-deployment, Grafana showed a spike in 5xx errors on the auth service in ApnaKart.

TASK

- Identify root cause, minimise user impact, restore service, and prevent recurrence.

ACTION

- Checked Prometheus/Grafana — 5xx spike correlated exactly with deployment timestamp.
- kubectl logs showed: 'Connection refused to RDS endpoint' — DB connectivity issue.
- Root cause: Terraform change had inadvertently tightened the RDS Security Group, blocking EKS node IPs.
- Immediate fix: kubectl rollout undo deployment/auth-service — restored in under 2 minutes.
- Root fix: corrected the Terraform SG rule, ran plan to validate, applied via pipeline.

RESULT

- < 3 minutes total downtime. Added SG validation check to the Terraform pipeline to prevent recurrence.

Interview tip: STAR format: Situation, Task, Action, Result. End with the prevention measure — shows you close the loop, not just fix the symptom.

Q3. Why should we hire you?

DIFFERENTIATION

- Not just familiar with the stack — have owned it end-to-end in production: Terraform, EKS, Jenkins, Prometheus, DevSecOps.
- Measurable impact: 77% faster deployments, 30% lower MTTR, zero critical vulnerabilities in production, 99.9% availability.
- Built both projects from zero infrastructure — architected decisions, not just executed instructions.
- Automation mindset: eliminated 70% of manual tasks through scripting — scale without headcount.
- Fast learner who has operated independently: designed, built, and ran production infrastructure during internship.

Interview tip: Lead with outcomes, not effort. Avoid 'I'm a hard worker, passionate about DevOps' — every candidate says that. Your numbers are specific and verifiable.

10. Smart Questions to Ask the Interviewer

Q1. Questions that demonstrate depth and genuine interest

ABOUT THE TECH STACK AND CHALLENGES

- 'What does your current Kubernetes setup look like, and what are the biggest pain points the team is working to solve?'
- 'How mature is your CI/CD pipeline today — what does a typical deployment to production look like?'
- 'How do you currently handle observability across microservices? Are there gaps you're looking to address?'

ABOUT THE ROLE

- 'What would a successful first 90 days look like for someone in this role?'
- 'What's the balance between greenfield work and maintenance/improvements on existing infrastructure?'
- 'How does the DevOps team collaborate with development teams — embedded, platform-as-a-service, or shared ownership?'

ABOUT THE TEAM AND CULTURE

- 'How does the team handle on-call rotations and incident response?'
- 'What does the growth path look like for a DevOps Engineer here?'

Interview tip: Ask 2-3 questions maximum. Avoid anything about salary, leave, or perks in the first round. Questions about tech challenges show you're thinking like a colleague, not a job-seeker.

Final Checklist Before Your Interview

- ✓ Practise every answer out loud — not by reading, by speaking
- ✓ Time your Tell me about yourself answer — it must be under 90 seconds
- ✓ For every technical answer: concept → your real example → metric
- ✓ **ELK Stack is on your resume — know at least one concrete use case**
- ✓ Prepare your 2-3 questions for the interviewer from Section 10
- ✓ **Your numbers are your proof: 77% · 99.9% · 35% · 40% · 70% · 30%**

Good luck, Sanket.